

SATQUERY AI

P1 — Siddharth

COMPLETE ENGINEERING PRD

Tech Lead + Backend Architect

Deadline: 20 September 2026 • Six developers in parallel • Contract-first microservices

This is an execution document, not an API-only specification. It defines ownership, repository locations, modules, issues, dependencies, tests, security, observability, daily execution and a ready-to-copy coding-agent prompt.

1. Role Mission

Field	Implementation specification
Primary owner	Gateway/BFF, Auth, org/RBAC, contracts, Mission Service
Backup/review	Mission backup; cross-service architecture
Architecture reviewer	P1
QA/DevOps	P6
Success condition	Owned subsystem is independently runnable, contract-conformant, observable, secure and integrated into the flagship flood mission.
Non-ownership	Other teams' private tables, internal implementation details, credentials and undocumented contract changes.

2. System Position

P1 system position



Communication: browser→Gateway uses HTTPS/WebSocket; service-to-service uses versioned APIs/events; long-running EO/ML/report/monitoring work is asynchronous; storage and schemas have explicit ownership.

3. Repository / Folder Ownership

Folder	Purpose	Must not contain
services/gateway/	Owned implementation, tests or docs	Unrelated service logic / credentials
services/mission/	Owned implementation, tests or docs	Unrelated service logic / credentials
packages/contracts/	Owned implementation, tests or docs	Unrelated service logic / credentials
packages/auth/	Owned implementation, tests or docs	Unrelated service logic / credentials
packages/shared/	Owned implementation, tests or docs	Unrelated service logic / credentials
docs/architecture/adr/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/contracts/	Owned implementation, tests or docs	Unrelated service logic / credentials

4. Technology Stack

Field	Implementation specification
Stack	Python/FastAPI/Pydantic/PostgreSQL+PostGIS/Redis/Celery/OAuth2-OIDC/JWT/Docker/OpenTelemetry
Why	Matches the supplied SatQuery architecture and the six-person playbook; additions are implementation recommendations, not product requirements.
Serialization	OpenAPI/JSON Schema; canonical DTOs/events.
Testing	pytest and contract/integration tests; frontend uses browser E2E where applicable.
Observability	OpenTelemetry + structured logs + Prometheus/Grafana.
Containerization	Docker; Kubernetes remains future/post-SIH.

5. Engineering Rules

- All internal APIs use /api/v1; breaking changes require versioning and P1 review.
- Canonical ErrorResponse: code, message, details[], retryable, trace_id.
- Propagate trace_id, mission_id, run_id, job_id, organization_id where applicable.
- organization_id comes from verified auth context, never a request body.
- No cross-service private DB reads.
- Long-running EO/ML/report/monitoring work returns 202 + job_id.
- Results retain dataset_id/observation_ids/model_version/processing_version as applicable.

- Idempotency-Key is used for duplicate-sensitive writes.
- Remote calls use bounded timeouts/retries/backoff; no retry storms.
- Never log secrets, auth headers or raw sensitive inputs.

6. Complete Issue Register

ID	Title	Pri	Type	Depends	Contact
P1-01	Monorepo and service skeletons	P0	Infrastructure	None	P6
P1-02	Freeze canonical API, error and event contracts	P0	Contract	P1-01	P2
P1-03	OAuth2/OIDC JWT verification foundation	P0	Security	P1-01	P5
P1-04	RBAC and tenant isolation	P0	Security	P1-03	P6
P1-05	API Gateway/BFF routing, CORS and rate limits	P0	Feature/Security	P1-04	P2
P1-06	Mission and AOI lifecycle service	P0	Feature	P1-05	P5
P1-07	Mission run/job lifecycle bridge	P0	Integration	P1-06,P6-02	P6
P1-08	Mission WebSocket status stream	P1	Feature	P1-07,P6-02	P2
P1-09	Audit trail and trace metadata	P0	Security	P1-08,P6-02	P5
P1-10	Shared internal client and error mapping	P0	Integration	P1-09,P6-02	P6
P1-11	Service-to-service identities and scopes	P0	Security	P1-10	P2
P1-12	Idempotency and duplicate execution controls	P0	Reliability	P1-11	P5
P1-13	Required architecture ADR set	P1	Architecture	P1-12	P6
P1-14	OpenAPI/event compatibility CI gate	P0	Testing/CI	P1-13	P2
P1-15	Mission ↔ Agent integration	P0	Integration	P1-14,P2-05	P5
P1-16	Generated frontend client verification	P0	Integration	P1-15,P2-05	P6
P1-17	Timeout/retry/circuit-breaker policy	P1	Reliability	P1-16,P2-05	P2
P1-18	Final architecture hardening and sign-off	P0	Release	P1-17,P2-05	P5

P1-01 — Monorepo and service skeletons

Field	Implementation specification
Priority	P0
Type	Infrastructure
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement monorepo and service skeletons as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for monorepo and service skeletons. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/Internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_monorepo_and_service_skeletons_valid()', 'test_monorepo_and_service_skeletons_invalid_input()']
Integration tests	['test_p1_01_service_boundary()']
Contract tests	['test_p1_01_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, monorepo and service skeletons completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	None
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-02
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run monorepo and service skeletons against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-02 — Freeze canonical API, error and event contracts

Field	Implementation specification
Priority	P0
Type	Contract
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement freeze canonical api, error and event contracts as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for freeze canonical api, error and event contracts. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_freeze_canonical_api_error_and_event_contracts_valid()', 'test_freeze_canonical_api_error_and_event_contracts_invalid_input()']
Integration tests	['test_p1_02_service_boundary()']
Contract tests	['test_p1_02_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, freeze canonical api, error and event contracts completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-03
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run freeze canonical api, error and event contracts against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-03 — OAuth2/OIDC JWT verification foundation

Field	Implementation specification
Priority	P0
Type	Security
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement oauth2/oidc jwt verification foundation as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for oauth2/oidc jwt verification foundation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_oauth2_oidc_jwt_verification_foundation_valid()', 'test_oauth2_oidc_jwt_verification_foundation_invalid_input()']
Integration tests	['test_p1_03_service_boundary()']
Contract tests	['test_p1_03_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, oauth2/oidc jwt verification foundation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-04
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run oauth2/oidc jwt verification foundation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-04 — RBAC and tenant isolation

Field	Implementation specification
Priority	P0
Type	Security
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement rbac and tenant isolation as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for rbac and tenant isolation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aais, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/Internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_rbac_and_tenant_isolation_valid()', 'test_rbac_and_tenant_isolation_invalid_input()']
Integration tests	['test_p1_04_service_boundary()']
Contract tests	['test_p1_04_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, rbac and tenant isolation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-03
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-05
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run rbac and tenant isolation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-05 — API Gateway/BFF routing, CORS and rate limits

Field	Implementation specification
Priority	P0
Type	Feature/Security
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement api gateway/bff routing, cors and rate limits as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for api gateway/bff routing, cors and rate limits. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aais, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_api_gateway_bff_routing_cors_and_rate_limits_valid()', 'test_api_gateway_bff_routing_cors_and_rate_limits_invalid_input()']
Integration tests	['test_p1_05_service_boundary()']
Contract tests	['test_p1_05_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, api gateway/bff routing, cors and rate limits completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-04
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-06
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run api gateway/bff routing, cors and rate limits against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-06 — Mission and AOI lifecycle service

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement mission and aoι lifecycle service as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission and aoι lifecycle service. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_and_aoι_lifecycle_service_valid()', 'test_mission_and_aoι_lifecycle_service_invalid_input()']
Integration tests	['test_p1_06_service_boundary()']
Contract tests	['test_p1_06_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission and aoι lifecycle service completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-07
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission and aoι lifecycle service against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-07 — Mission run/job lifecycle bridge

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement mission run/job lifecycle bridge as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission run/job lifecycle bridge. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_run_job_lifecycle_bridge_valid()', 'test_mission_run_job_lifecycle_bridge_invalid_input()']
Integration tests	['test_p1_07_service_boundary()']
Contract tests	['test_p1_07_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission run/job lifecycle bridge completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-06, P6-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-08
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission run/job lifecycle bridge against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-08 — Mission WebSocket status stream

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement mission websocket status stream as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission websocket status stream. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_websocket_status_stream_valid()', 'test_mission_websocket_status_stream_invalid_input()']
Integration tests	['test_p1_08_service_boundary()']
Contract tests	['test_p1_08_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission websocket status stream completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-07, P6-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-09
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission websocket status stream against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-09 — Audit trail and trace metadata

Field	Implementation specification
Priority	P0
Type	Security
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement audit trail and trace metadata as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for audit trail and trace metadata. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_audit_trail_and_trace_metadata_valid()', 'test_audit_trail_and_trace_metadata_invalid_input()']
Integration tests	['test_p1_09_service_boundary()']
Contract tests	['test_p1_09_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, audit trail and trace metadata completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-08, P6-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-10
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run audit trail and trace metadata against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-10 — Shared internal client and error mapping

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement shared internal client and error mapping as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for shared internal client and error mapping. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aos, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/Internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_shared_internal_client_and_error_mapping_valid()', 'test_shared_internal_client_and_error_mapping_invalid_input()']
Integration tests	['test_p1_10_service_boundary()']
Contract tests	['test_p1_10_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, shared internal client and error mapping completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-09,P6-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-11
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run shared internal client and error mapping against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-11 — Service-to-service identities and scopes

Field	Implementation specification
Priority	P0
Type	Security
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement service-to-service identities and scopes as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for service-to-service identities and scopes. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_service_to_service_identities_and_scopes_valid()', 'test_service_to_service_identities_and_scopes_invalid_input()']
Integration tests	['test_p1_11_service_boundary()']
Contract tests	['test_p1_11_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, service-to-service identities and scopes completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-10
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-12
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run service-to-service identities and scopes against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-12 — Idempotency and duplicate execution controls

Field	Implementation specification
Priority	P0
Type	Reliability
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement idempotency and duplicate execution controls as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for idempotency and duplicate execution controls. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_idempotency_and_duplicate_execution_controls_valid()', 'test_idempotency_and_duplicate_execution_controls_invalid_input()']
Integration tests	['test_p1_12_service_boundary()']
Contract tests	['test_p1_12_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, idempotency and duplicate execution controls completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-11
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-13
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run idempotency and duplicate execution controls against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-13 — Required architecture ADR set

Field	Implementation specification
Priority	P1
Type	Architecture
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement required architecture adr set as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for required architecture adr set. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_required_architecture_adr_set_valid()', 'test_required_architecture_adr_set_invalid_input()']
Integration tests	['test_p1_13_service_boundary()']
Contract tests	['test_p1_13_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, required architecture adr set completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-12
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-14
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run required architecture adr set against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-14 — OpenAPI/event compatibility CI gate

Field	Implementation specification
Priority	P0
Type	Testing/CI
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement openapi/event compatibility ci gate as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for openapi/event compatibility ci gate. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_openapi_event_compatibility_ci_gate_valid()', 'test_openapi_event_compatibility_ci_gate_invalid_input()']
Integration tests	['test_p1_14_service_boundary()']
Contract tests	['test_p1_14_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, openapi/event compatibility ci gate completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-13
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-15
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run openapi/event compatibility ci gate against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-15 — Mission ↔ Agent integration

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement mission ↔ agent integration as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission ↔ agent integration. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aais, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_agent_integration_valid()', 'test_mission_agent_integration_invalid_input()']
Integration tests	['test_p1_15_service_boundary()']
Contract tests	['test_p1_15_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission ↔ agent integration completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-14,P2-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-16
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission ↔ agent integration against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-16 — Generated frontend client verification

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P1
Reviewer	P6
Cross-service contact	P6
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement generated frontend client verification as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for generated frontend client verification. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_generated_frontend_client_verification_valid()', 'test_generated_frontend_client_verification_invalid_input()']
Integration tests	['test_p1_16_service_boundary()']
Contract tests	['test_p1_16_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, generated frontend client verification completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-15, P2-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-17
Parallelizable with	P6 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P6. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run generated frontend client verification against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-17 — Timeout/retry/circuit-breaker policy

Field	Implementation specification
Priority	P1
Type	Reliability
Owner	P1
Reviewer	P6
Cross-service contact	P2
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement timeout/retry/circuit-breaker policy as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for timeout/retry/circuit-breaker policy. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aosis, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_timeout_retry_circuit_breaker_policy_valid()', 'test_timeout_retry_circuit_breaker_policy_invalid_input()']
Integration tests	['test_p1_17_service_boundary()']
Contract tests	['test_p1_17_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, timeout/retry/circuit-breaker policy completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-16,P2-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P1-18
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run timeout/retry/circuit-breaker policy against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P1-18 — Final architecture hardening and sign-off

Field	Implementation specification
Priority	P0
Type	Release
Owner	P1
Reviewer	P6
Cross-service contact	P5
Service	Gateway / Mission
Folders	services/gateway/; services/mission/; packages/contracts/; packages/auth/; packages/shared/; docs/architecture/adr/; tests/contracts/
Objective	Implement final architecture hardening and sign-off as a production-shaped module inside the P1 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for final architecture hardening and sign-off. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/gateway/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/missions, /api/v1/aois, /api/v1/health, /ws/v1/missions/{id} as applicable
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_final_architecture_hardening_and_sign_off_valid()', 'test_final_architecture_hardening_and_sign_off_invalid_input()']
Integration tests	['test_p1_18_service_boundary()']
Contract tests	['test_p1_18_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, final architecture hardening and sign-off completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-17,P2-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	Release/integration gate
Parallelizable with	P5 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P1. Consumer/contact: P5. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run final architecture hardening and sign-off against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

7. Test Plan

The issue-level test names below are the minimum required suite. Expand edge cases discovered during implementation.

P1-01 — Monorepo and service skeletons

- `test_monorepo_and_service_skeletons_valid()`
- `test_monorepo_and_service_skeletons_invalid_input()`
- `test_p1_01_service_boundary()`
- `test_p1_01_schema_compatibility()`

P1-02 — Freeze canonical API, error and event contracts

- `test_freeze_canonical_api_error_and_event_contracts_valid()`
- `test_freeze_canonical_api_error_and_event_contracts_invalid_input()`
- `test_p1_02_service_boundary()`
- `test_p1_02_schema_compatibility()`

P1-03 — OAuth2/OIDC JWT verification foundation

- `test_oauth2_oidc_jwt_verification_foundation_valid()`
- `test_oauth2_oidc_jwt_verification_foundation_invalid_input()`
- `test_p1_03_service_boundary()`
- `test_p1_03_schema_compatibility()`

P1-04 — RBAC and tenant isolation

- `test_rbac_and_tenant_isolation_valid()`
- `test_rbac_and_tenant_isolation_invalid_input()`
- `test_p1_04_service_boundary()`
- `test_p1_04_schema_compatibility()`

P1-05 — API Gateway/BFF routing, CORS and rate limits

- `test_api_gateway_bff_routing_cors_and_rate_limits_valid()`
- `test_api_gateway_bff_routing_cors_and_rate_limits_invalid_input()`
- `test_p1_05_service_boundary()`
- `test_p1_05_schema_compatibility()`

P1-06 — Mission and AOI lifecycle service

- `test_mission_and_aoi_lifecycle_service_valid()`
- `test_mission_and_aoi_lifecycle_service_invalid_input()`
- `test_p1_06_service_boundary()`
- `test_p1_06_schema_compatibility()`

P1-07 — Mission run/job lifecycle bridge

- `test_mission_run_job_lifecycle_bridge_valid()`
- `test_mission_run_job_lifecycle_bridge_invalid_input()`
- `test_p1_07_service_boundary()`
- `test_p1_07_schema_compatibility()`

P1-08 — Mission WebSocket status stream

- `test_mission_websocket_status_stream_valid()`
- `test_mission_websocket_status_stream_invalid_input()`
- `test_p1_08_service_boundary()`
- `test_p1_08_schema_compatibility()`

P1-09 — Audit trail and trace metadata

- `test_audit_trail_and_trace_metadata_valid()`
- `test_audit_trail_and_trace_metadata_invalid_input()`
- `test_p1_09_service_boundary()`
- `test_p1_09_schema_compatibility()`

P1-10 — Shared internal client and error mapping

- `test_shared_internal_client_and_error_mapping_valid()`
- `test_shared_internal_client_and_error_mapping_invalid_input()`
- `test_p1_10_service_boundary()`
- `test_p1_10_schema_compatibility()`

P1-11 — Service-to-service identities and scopes

- `test_service_to_service_identities_and_scopes_valid()`
- `test_service_to_service_identities_and_scopes_invalid_input()`
- `test_p1_11_service_boundary()`
- `test_p1_11_schema_compatibility()`

P1-12 — Idempotency and duplicate execution controls

- `test_idempotency_and_duplicate_execution_controls_valid()`
- `test_idempotency_and_duplicate_execution_controls_invalid_input()`
- `test_p1_12_service_boundary()`
- `test_p1_12_schema_compatibility()`

P1-13 — Required architecture ADR set

- `test_required_architecture_adr_set_valid()`
- `test_required_architecture_adr_set_invalid_input()`
- `test_p1_13_service_boundary()`
- `test_p1_13_schema_compatibility()`

P1-14 — OpenAPI/event compatibility CI gate

- `test_openapi_event_compatibility_ci_gate_valid()`
- `test_openapi_event_compatibility_ci_gate_invalid_input()`
- `test_p1_14_service_boundary()`
- `test_p1_14_schema_compatibility()`

P1-15 — Mission ↔ Agent integration

- `test_mission_agent_integration_valid()`
- `test_mission_agent_integration_invalid_input()`
- `test_p1_15_service_boundary()`
- `test_p1_15_schema_compatibility()`

P1-16 — Generated frontend client verification

- `test_generated_frontend_client_verification_valid()`
- `test_generated_frontend_client_verification_invalid_input()`
- `test_p1_16_service_boundary()`
- `test_p1_16_schema_compatibility()`

P1-17 — Timeout/retry/circuit-breaker policy

- `test_timeout_retry_circuit_breaker_policy_valid()`
- `test_timeout_retry_circuit_breaker_policy_invalid_input()`
- `test_p1_17_service_boundary()`
- `test_p1_17_schema_compatibility()`

P1-18 — Final architecture hardening and sign-off

- test_final_architecture_hardening_and_sign_off_valid()
- test_final_architecture_hardening_and_sign_off_invalid_input()
- test_p1_18_service_boundary()
- test_p1_18_schema_compatibility()

8. Security Implementation

Control	Implementation owner	Required behavior
Authentication	P1	Implement/test where applicable; P6 supplies regression gate.
RBAC + tenant isolation	P1	Implement/test where applicable; P6 supplies regression gate.
Input/schema validation	P1	Implement/test where applicable; P6 supplies regression gate.
GeoJSON/raster validation	P1	Implement/test where applicable; P6 supplies regression gate.
SSRF/provider allowlist	P1	Implement/test where applicable; P6 supplies regression gate.
Prompt injection	P1	Implement/test where applicable; P6 supplies regression gate.
AI tool permissions	P1	Implement/test where applicable; P6 supplies regression gate.
Rate/request limits	P1	Implement/test where applicable; P6 supplies regression gate.
Secrets/TLS	P1	Implement/test where applicable; P6 supplies regression gate.
Audit logging	P1	Implement/test where applicable; P6 supplies regression gate.
Dependency/container scanning	P1	Implement/test where applicable; P6 supplies regression gate.

9. Observability

Required identifiers: trace_id, mission_id, run_id, job_id, organization_id, model_version, dataset_id. Metrics should expose latency, queue time, errors and user-visible mission state. Role metrics: gateway_requests_total, auth_fail_total, rbac_denied_total, mission_state_transition_total

10. Failure Modes

Failure	Detection	Recovery/fallback	User behavior
Bhoonidhi unavailable	timeout/5xx	bounded retry → pinned permitted fixture	show fallback clearly
Inference timeout/OOM	job timeout/worker exit	bounded retry → deterministic fixture for demo	show explicit degraded/failure state
Redis restart	connection/requeue failure	idempotent requeue/reconciliation	mission remains recoverable
Invalid AOI/raster	validator rejection	no expensive processing	show actionable validation
Low confidence	confidence gate	clarify/acquire/explicit uncertainty	never confident prose
Tenant violation	RBAC denial	no retry/no leakage	403/404

11. Implementation Order

Foundation → core domain → cross-service integration → differentiation → hardening/release. Earlier issues unlock later ones, but frozen contracts and fixtures allow parallel implementation. Do not serialize on live dependencies. Use mocks whenever the producer is unfinished.

12. Daily Execution Plan — Sep 11–20

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 11	P1-01, P1-02	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 12	P1-02, P1-03	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 13	P1-03, P1-04	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 14	P1-04, P1-05	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 15	P1-05, P1-06	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 16	P1-06, P1-07	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 17	P1-07, P1-08	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 18	P1-08, P1-09	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 19	P1-09, P1-10	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 20	P1-10, P1-11	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

13. Definition of Ready / Done

Field	Implementation specification
DoR	Requirements clear; owner/reviewer; contract; dependencies; acceptance; tests; demo result defined.
DoD	Merged; unit/contract/integration tests; security; telemetry; docs/schema; DEV/DEMO; acceptance demonstrated.

14. Cross-Team Communication

- P1 is architecture/contract authority.
- P6 is CI/DEV/DEMO/release authority.
- P2↔P3: sensor arbitration, inference and evidence.
- P2↔P4: observation selection, data quality and acquisition.
- P3↔P4: AssetRef, CRS/resolution and preprocessing.
- P4↔P5: tiles, masks, GeoJSON and map performance.
- P5↔P1: generated client and browser DTOs.
- Escalate blockers after 30–60 minutes; use contract mocks/fixtures instead of waiting.

15. Prompt to Give to the Coding Agent

You are P1 implementation agent. Own gateway, mission, auth/RBAC, contracts and architecture. Inspect repository/contracts first. Implement the assigned issue end-to-end; never trust body tenant IDs; use verified AuthContext; use canonical ErrorResponse, idempotency and trace propagation. Do not read private tables of other services. Update OpenAPI/events before breaking changes. Add unit, contract, integration and security tests; run lint/type/unit/contract/integration; report changed files, contracts, migrations, security and blockers.

16. Role Release Checklist

- [] All P0 issues merged or explicitly waived.
- [] Unit + contract + relevant integration tests green.
- [] Security controls and negative tests green.
- [] Trace/metrics/logging present.
- [] DEV/DEMO verified.
- [] Failure behavior demonstrated.
- [] Fixtures/model versions pinned where applicable.
- [] Documentation/ADR updated.
- [] Role-specific flagship demo path works.